PYTHON PROGRAMMING ACTIVITY-BASED LEARNING

SELECTED 10 CASE STUDIES — REPORT

Student Name	B Charishma sri sai
Roll Number	26001382
Course	Python Programming
Academic Year	2026–2027

Each case study includes a problem statement, objectives, assumptions, algorithm, pseudocode, Python source code, terminal-style sample output, test cases, flow sequence, and conclusion.

Prepared using the supplied case-study list and the first PDF as the report-format reference.

CONTENTS

Selected case studies from the supplied Python Activity-Based Learning list:

Case Study 7 — Retail Billing with Multiple Discount Rules

Case Study 18 — Fuel Consumption and Route Comparison

Case Study 27 — Advanced Numeric Pattern Generator

Case Study 39 — Multi-Stage Examination Eligibility System

Case Study 45 — Weekly Electricity Consumption Matrix

Case Study 34 — Student Feedback Sentiment Scoring

Case Study 65 — Campus Resource Allocation Under Constraints

Case Study 76 — University Examination Result Moderation

Case Study 87 — Examination Question Difficulty and Performance Analysis

Case Study 93 — Smart Public Transport Fare Reconciliation

Project Objectives

• Understand practical problems using Python programming.

• Develop logical and computational thinking.

• Represent solutions using algorithms, pseudocode, and flow sequences.

• Implement and test programs using the assigned syllabus topics.

• Analyse program outputs and draw conclusions.

Note: Output panels are terminal-style sample-run renderings for readability; they are not photographs of an external IDE.

CASE STUDY 7 — Retail Billing with Multiple Discount Rules

Question / Problem Statement

Process multiple products and quantities, apply category-specific taxes and threshold discounts subject to a maximum concession, and generate an itemized invoice.

Objectives

• Calculate line totals from unit price and quantity.

• Apply a threshold discount and cap the discount amount.

• Calculate category-specific tax and print an itemized bill.

Assumptions

• Each item is stored as (name, category, unit_price, quantity).

• Electronics tax is 18%; groceries tax is 5%.

• A 10% discount applies when the pre-discount subtotal is at least Rs. 2,000; discount is capped at Rs. 300.

• Tax is calculated after discount and uses each item's category rate.

Algorithm

1. Store item records and tax rates.

2. Calculate each line total and the subtotal.

3. If the subtotal meets the threshold, calculate 10% discount and cap it at Rs. 300.

4. Subtract the discount proportionally from the subtotal for the taxable amount; calculate tax by category on each discounted line.

5. Print the itemized invoice, discount, tax, and grand total.

Pseudocode

```
START
Read item records and tax rates
Calculate line totals and subtotal
IF subtotal >= 2000: discount = minimum(10% of subtotal, 300)
Apply discount proportionally; calculate category tax
Display invoice and grand total
END
```

Python Program

```
items = [
    ("Headphones", "electronics", 1200, 1),
    ("Notebook", "groceries", 100, 4),
    ("Mouse", "electronics", 500, 1)
]
tax_rates = {"electronics": 0.18, "groceries": 0.05}
subtotal = sum(price * qty for name, cat, price, qty in items)
discount = min(subtotal * 0.10, 300) if subtotal >= 2000 else 0
factor = (subtotal - discount) / subtotal if subtotal else 1
tax_total = 0

print("ITEMIZED INVOICE")
for name, category, price, qty in items:
    line = price * qty
    discounted_line = line * factor
    tax = discounted_line * tax_rates[category]
    tax_total += tax
    print(f"{name}: {qty} x Rs.{price} = Rs.{line:.2f}")
print(f"Subtotal: Rs.{subtotal:.2f}")
print(f"Discount: Rs.{discount:.2f}")
print(f"Tax: Rs.{tax_total:.2f}")
print(f"Grand total: Rs.{subtotal - discount + tax_total:.2f}")
```

CASE STUDY 7 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
ITEMIZED INVOICE
Headphones: 1 x Rs.1200 = Rs.1200.00
Notebook: 4 x Rs.100 = Rs.400.00
Mouse: 1 x Rs.500 = Rs.500.00
Subtotal: Rs.2100.00
Discount: Rs.210.00
Tax: Rs.293.40
Grand total: Rs.2183.40
```

Test Cases

Input / Condition	Expected Result
Subtotal Rs. 2,100	10% discount applied: Rs. 210
Subtotal below Rs. 2,000	No threshold discount
Discount exceeds Rs. 300	Discount limited to Rs. 300

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The invoice program combines quantity-based totals, a capped threshold discount, and category-specific taxes to produce a transparent final bill.

CASE STUDY 18 — Fuel Consumption and Route Comparison

Question / Problem Statement

Calculate fuel needs and journey costs for several vehicles using distance, mileage, fuel price, and a reserve requirement, then identify the lowest-cost feasible option.

Objectives

• Estimate fuel required for each vehicle.

• Include a reserve quantity in feasibility checks.

• Compare total journey costs and select the cheapest feasible vehicle.

Assumptions

• Journey distance is 300 km and fuel price is Rs. 105 per litre.

• A 2-litre reserve must remain after the trip; available fuel is 25 litres.

• Vehicle records contain (name, mileage_km_per_litre, available_fuel_litres).

Algorithm

1. Set journey distance, fuel price, and required reserve.

2. For each vehicle, calculate trip fuel as distance divided by mileage.

3. Add the reserve requirement to obtain required fuel.

4. Mark a vehicle feasible if available fuel is at least the required fuel.

5. Calculate trip cost and choose the lowest-cost feasible vehicle.

Pseudocode

```
START
Set distance, fuel price and reserve
FOR each vehicle: fuel_needed = distance / mileage
Check available fuel >= fuel_needed + reserve
If feasible, calculate trip cost and compare
Display feasible options and cheapest vehicle
END
```

Python Program

```
distance = 300
fuel_price = 105
reserve = 2
vehicles = [
    ("Car A", 15, 25),
    ("Car B", 20, 18),
    ("Car C", 12, 30)
]
best = None

for name, mileage, available in vehicles:
    fuel_needed = distance / mileage
    total_required = fuel_needed + reserve
    cost = fuel_needed * fuel_price
    feasible = available >= total_required
    status = "FEASIBLE" if feasible else "NOT FEASIBLE"
    print(f"{name}: fuel={fuel_needed:.2f} L, "
          f"cost=Rs.{cost:.2f}, {status}")
    if feasible and (best is None or cost < best[1]):
        best = (name, cost)

if best:
    print("Lowest-cost feasible vehicle:", best[0])
    print(f"Trip fuel cost: Rs.{best[1]:.2f}")
else:
    print("No vehicle meets the reserve requirement.")
```

CASE STUDY 18 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Car A: fuel=20.00 L, cost=Rs.2100.00, FEASIBLE
Car B: fuel=15.00 L, cost=Rs.1575.00, FEASIBLE
Car C: fuel=25.00 L, cost=Rs.2625.00, FEASIBLE
Lowest-cost feasible vehicle: Car B
Trip fuel cost: Rs.1575.00
```

Test Cases

Input / Condition	Expected Result
Car B: 18 L available	Feasible; 15 L trip fuel + 2 L reserve
Car C: 30 L available	Feasible because 25 L trip fuel + 2 L reserve is within 30 L
No vehicle meets requirement	Display no feasible vehicle

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The comparison calculates fuel and cost for each option and chooses the least expensive vehicle that can complete the journey while preserving the required reserve.

CASE STUDY 27 — Advanced Numeric Pattern Generator

Question / Problem Statement

Generate a hollow diamond numerical pattern for a supplied size, validate invalid dimensions, and keep the output aligned.

Objectives

• Validate the requested pattern size.

• Use nested loops to print a symmetric hollow diamond.

• Display a clear message for invalid input.

Assumptions

• The size must be an integer from 1 through 9.

• The pattern uses row numbers on the left and right edges with spaces inside.

• For size 1, the output is a single 1.

Algorithm

1. Read the size and validate its range.

2. Build the top half from row 1 through size.

3. Print leading spaces, the row number, and an inner gap for non-first rows.

4. Build the bottom half in reverse order.

5. Display the complete pattern.

Pseudocode

```
START
Read size n
IF n < 1 OR n > 9: print invalid size
ELSE print top half using nested loops
Print bottom half in reverse order
END
```

Python Program

```
n = 5
if not isinstance(n, int) or n < 1 or n > 9:
    print("Invalid size: enter an integer from 1 to 9")
else:
    for row in range(1, n + 1):
        print(" " * (n - row), end="")
        if row == 1:
            print(1)
        else:
            print(row, " " * (2 * row - 3), row, sep="")
    for row in range(n - 1, 0, -1):
        print(" " * (n - row), end="")
        if row == 1:
            print(1)
        else:
            print(row, " " * (2 * row - 3), row, sep="")
```

CASE STUDY 27 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
    1
   2 2
  3   3
 4     4
5       5
 4     4
  3   3
   2 2
    1
```

Test Cases

Input / Condition	Expected Result
n = 5	Prints a symmetric hollow diamond
n = 1	Prints a single 1
n = 0 or n = 10	Invalid-size message

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The nested-loop solution produces a symmetric hollow diamond and checks the dimension before generating the pattern.

CASE STUDY 39 — Multi-Stage Examination Eligibility System

Question / Problem Statement

Determine final-examination eligibility from attendance, laboratory completion, internal assessment, and fee-clearance conditions, reporting every unmet requirement.

Objectives

• Check each eligibility condition independently.

• Report all reasons for ineligibility rather than stopping at the first failure.

• Print a final eligible or not-eligible decision.

Assumptions

• Minimum attendance is 75%.

• All required laboratory work must be complete.

• Internal assessment must be at least 40 out of 100.

• Fees must be cleared.

Algorithm

1. Set a student's attendance, lab status, internal marks, and fee status.

2. Create an empty list of unmet requirements.

3. Check attendance, laboratory completion, internal marks, and fee clearance.

4. Append a message for each failed condition.

5. Print all messages and the final decision.

Pseudocode

```
START
Read eligibility details
Create empty unmet-requirements list
Check attendance, labs, internal marks, and fees
Add a message for each failed condition
IF no unmet requirements: eligible; ELSE: not eligible
END
```

Python Program

```
student =  B Charishma sri sai
attendance
labs_complete = True
internal_marks = 36
fees_cleared = False

unmet = []
if attendance < 75:
    unmet.append("Attendance below 75%")
if not labs_complete:
    unmet.append("Laboratory work incomplete")
if internal_marks < 40:
    unmet.append("Internal marks below 40")
if not fees_cleared:
    unmet.append("Fees not cleared")

print("Student:", student)
if unmet:
    print("NOT ELIGIBLE")
    for reason in unmet:
        print("-", reason)
else:
    print("ELIGIBLE for final examination")
```

CASE STUDY 39 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Student: B Charishma sri sai
NOT ELIGIBLE
- Internal marks below 40
- Fees not cleared
```

Test Cases

Input / Condition	Expected Result
Attendance 82%, labs complete, marks 45, fees cleared	Eligible
Attendance 70%	Attendance requirement reported
Marks 36 and fees not cleared	Both failures reported

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The system checks every stage independently and provides an actionable list of unmet examination requirements.

CASE STUDY 45 — Weekly Electricity Consumption Matrix

Question / Problem Statement

Process appliance-wise electricity readings across seven days, calculate daily and appliance totals, and identify consumption that exceeds specified limits.

Objectives

• Store readings in a nested list.

• Calculate daily and appliance-wise totals.

• Flag daily totals above the defined limit.

Assumptions

• Rows represent appliances and columns represent Monday through Sunday.

• Values are kWh readings for the example week.

• The daily alert limit is 12 kWh.

Algorithm

1. Store appliance names and a 7-day reading matrix.

2. For each day, sum the readings from all appliances.

3. For each appliance, sum its seven readings.

4. Flag any daily total above the limit.

5. Display daily totals, appliance totals, and alerts.

Pseudocode

```
START
Read appliance names and nested reading list
FOR each day: sum appliance readings
FOR each appliance: sum seven daily readings
Flag daily total > limit
Display totals and warnings
END
```

Python Program

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
appliances = ["Fan", "Lights", "Computer"]
readings = [
    [2, 2, 2, 2, 2, 3, 3],
    [1, 1, 1, 1, 1, 2, 2],
    [3, 3, 4, 4, 3, 5, 5]
]
daily_limit = 7

daily_totals = []
for col, day in enumerate(days):
    total = sum(row[col] for row in readings)
    daily_totals.append(total)
    status = " ALERT" if total > daily_limit else ""
    print(f"{day}: {total} kWh{status}")

print("Appliance totals:")
for name, row in zip(appliances, readings):
    print(f"{name}: {sum(row)} kWh")
```

CASE STUDY 45 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Mon: 6 kWh
Tue: 6 kWh
Wed: 7 kWh
Thu: 7 kWh
Fri: 6 kWh
Sat: 10 kWh ALERT
Sun: 10 kWh ALERT
Appliance totals:
Fan: 16 kWh
Lights: 9 kWh
Computer: 27 kWh
```

Test Cases

Input / Condition	Expected Result
Daily total = 6 kWh	No alert
Daily total = 10 kWh	Alert because total exceeds 7 kWh
Appliance row [1,1,1,1,1,1,1]	Weekly total = 7 kWh

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The nested-list matrix summarizes consumption by day and appliance and identifies days that exceed the chosen daily limit.

CASE STUDY 34 — Student Feedback Sentiment Scoring

Question / Problem Statement

Analyze feedback sentences using a supplied keyword list, count positive and negative indicators, calculate sentiment scores, and classify feedback using fixed thresholds.

Objectives

• Normalize feedback text before checking keywords.

• Count positive and negative words.

• Calculate and classify a simple sentiment score.

Assumptions

• Positive keywords: good, helpful, clear, excellent.

• Negative keywords: poor, confusing, slow, difficult.

• Score equals positive matches minus negative matches; score above 0 is Positive, below 0 is Negative, and 0 is Neutral.

Algorithm

1. Store feedback sentences and keyword lists.

2. Convert each sentence to lowercase and split into words.

3. Count positive and negative keyword matches.

4. Calculate score = positive count minus negative count.

5. Print counts, score, and sentiment label.

Pseudocode

```
START
Set positive and negative keyword lists
FOR each feedback sentence: normalize and split words
Count positive and negative matches
Calculate score = positive - negative
Classify score as Positive, Negative, or Neutral
END
```

Python Program

```
feedback = [
    "The lesson was clear and helpful",
    "The pace was slow and confusing",
    "The class was good"
]
positive_words = ["good", "helpful", "clear", "excellent"]
negative_words = ["poor", "confusing", "slow", "difficult"]

for sentence in feedback:
    words = sentence.lower().replace(".", "").split()
    positive = sum(word in positive_words for word in words)
    negative = sum(word in negative_words for word in words)
    score = positive - negative
    if score > 0:
        label = "Positive"
    elif score < 0:
        label = "Negative"
    else:
        label = "Neutral"
    print(f"Feedback: {sentence}")
    print(f"Positive={positive}, Negative={negative}, "
          f"Score={score}, Sentiment={label}")
```

CASE STUDY 34 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Feedback: The lesson was clear and helpful
Positive=2, Negative=0, Score=2, Sentiment=Positive
Feedback: The pace was slow and confusing
Positive=0, Negative=2, Score=-2, Sentiment=Negative
Feedback: The class was good
Positive=1, Negative=0, Score=1, Sentiment=Positive
```

Test Cases

Input / Condition	Expected Result
Contains clear and helpful	Score 2; Positive
Contains slow and confusing	Score -2; Negative
Contains no listed keywords	Score 0; Neutral

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The keyword-based method offers a simple, explainable sentiment score for sample feedback; it does not infer meaning beyond the supplied word lists.

CASE STUDY 65 — Campus Resource Allocation Under Constraints

Question / Problem Statement

Compare equipment requirements with available resources across activities, calculate shortages, and produce an allocation using a specified priority order.

Objectives

• Track the available quantity of each resource.

• Allocate resources to activities in priority order.

• Report fulfilled allocations and shortages.

Assumptions

• Resource stock is limited to projectors and laptops.

• Activities are processed in the order listed, which represents priority.

• An activity receives all requested quantities only if both resource needs can be met; otherwise no resources are allocated to that activity.

Algorithm

1. Set resource stock and prioritized activity requirements.

2. For each activity, compare requested resources with remaining stock.

3. If all requests can be met, allocate and subtract stock.

4. Otherwise record the unmet requirement and leave stock unchanged.

5. Display allocation results and remaining stock.

Pseudocode

```
START
Set stock and priority-ordered activity requests
FOR each activity: check every resource requirement
IF all requirements available: allocate and reduce stock
ELSE report shortage and do not allocate
Display allocation and remaining resources
END
```

Python Program

```
stock = {"projector": 3, "laptop": 10}
activities = [
    ("Workshop A", {"projector": 1, "laptop": 5}),
    ("Seminar B", {"projector": 2, "laptop": 4}),
    ("Club C", {"projector": 1, "laptop": 3})
]

for name, need in activities:
    shortages = []
    for resource, quantity in need.items():
        if quantity > stock.get(resource, 0):
            shortages.append(resource)
    if shortages:
        print(name, "NOT ALLOCATED; shortage:", ", ".join(shortages))
    else:
        for resource, quantity in need.items():
            stock[resource] -= quantity
        print(name, "ALLOCATED:", need)

print("Remaining stock:", stock)
```

CASE STUDY 65 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Workshop A ALLOCATED: {'projector': 1, 'laptop': 5}
Seminar B ALLOCATED: {'projector': 2, 'laptop': 4}
Club C NOT ALLOCATED; shortage: projector, laptop
Remaining stock: {'projector': 0, 'laptop': 1}
```

Test Cases

Input / Condition	Expected Result
Need within available stock	Allocation succeeds and stock decreases
Need exceeds projector stock	Activity not allocated
Unallocated activity	Stock remains unchanged

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The priority-ordered allocator respects resource limits and reports shortages without partially allocating an activity.

CASE STUDY 76 — University Examination Result Moderation

Question / Problem Statement

Process marks across multiple subjects, apply a specified moderation rule, preserve maximum-mark constraints, and compare original and moderated results.

Objectives

• Apply moderation only to eligible marks.

• Ensure no mark exceeds the subject maximum.

• Compare original and moderated totals.

Assumptions

• Each subject is out of 100.

• A student receives 5 moderation marks when the original mark is between 35 and 39 inclusive.

• Moderated marks are capped at 100.

• The rule is illustrative and is applied independently to each subject.

Algorithm

1. Store subject marks and the moderation rule.

2. For each subject, check whether the mark is in the eligible interval.

3. Add 5 marks if eligible and cap at 100.

4. Print original and moderated marks per subject.

5. Calculate and display original and moderated totals.

Pseudocode

```
START
Read subject marks
FOR each subject: if 35 <= mark <= 39, add 5
Cap moderated mark at 100
Accumulate original and moderated totals
Display subject comparison and totals
END
```

Python Program

```
marks = {"Python": 38, "Maths": 72, "English": 35, "Physics": 98}
original_total = 0
moderated_total = 0

print("SUBJECT  ORIGINAL  MODERATED")
for subject, mark in marks.items():
    moderated = mark + 5 if 35 <= mark <= 39 else mark
    moderated = min(moderated, 100)
    original_total += mark
    moderated_total += moderated
    print(f"{subject:<8} {mark:>5} {moderated:>10}")

print("Original total:", original_total)
print("Moderated total:", moderated_total)
```

CASE STUDY 76 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
SUBJECT  ORIGINAL  MODERATED
Python      38         43
Maths       72         72
English     35         40
Physics     98         98
Original total: 243
Moderated total: 253
```

Test Cases

Input / Condition	Expected Result
Mark 38	Moderated to 43
Mark 72	Unchanged at 72
Mark 98	Remains 98 and does not exceed 100

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The moderation routine applies the stated eligibility band, enforces the maximum mark, and makes the effect on the overall total visible.

CASE STUDY 87 — Examination Question Difficulty and Performance Analysis

Question / Problem Statement

Compare student answers against an answer key, calculate question-wise success rates, identify the most frequently missed questions, and classify question performance.

Objectives

• Count correct responses for each question.

• Calculate success rates as percentages.

• Identify the lowest-performing question and classify question difficulty.

Assumptions

• Answers are represented as lists in the same question order as the answer key.

• Comparison ignores letter case and surrounding spaces.

• A success rate below 50% is classified as Difficult; otherwise it is Standard.

Algorithm

1. Store the answer key and student answer lists.

2. For each question position, count matching answers.

3. Calculate success rate = correct responses / number of students × 100.

4. Identify the question with the lowest success rate.

5. Print each question's rate and difficulty label.

Pseudocode

```
START
Set answer key and student responses
FOR each question: count normalized matching answers
Calculate success percentage
Classify below 50% as Difficult; otherwise Standard
Find question with minimum success rate
Display analysis
END
```

Python Program

```
answer_key = ["A", "C", "B", "D"]
responses = [
    ["A", "C", "B", "D"],
    ["A", "B", "B", "C"],
    ["A", "C", "D", "C"],
    ["B", "C", "B", "D"]
]
rates = []
print("Question-wise performance")
for q in range(len(answer_key)):
    correct = sum(
        row[q].strip().upper() == answer_key[q]
        for row in responses
    )
    rate = correct / len(responses) * 100
    rates.append(rate)
    level = "Difficult" if rate < 50 else "Standard"
    print(f"Q{q + 1}: {correct}/{len(responses)} "
          f"correct, {rate:.0f}%, {level}")

hardest = rates.index(min(rates)) + 1
print("Most frequently missed question: Q", hardest)
```

CASE STUDY 87 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Question-wise performance
Q1: 3/4 correct, 75%, Standard
Q2: 3/4 correct, 75%, Standard
Q3: 3/4 correct, 75%, Standard
Q4: 1/4 correct, 25%, Difficult
Most frequently missed question: Q 4
```

Test Cases

Input / Condition	Expected Result
All students answer correctly	100% success rate
Fewer than half answer correctly	Difficult classification
Tie for lowest rate	First lowest-rate question is reported by index

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The program calculates question-level success rates and identifies a lowest-performing question; tied minimum rates are resolved by selecting the first one.

CASE STUDY 93 — Smart Public Transport Fare Reconciliation

Question / Problem Statement

Calculate passenger fares across multiple routes and passenger categories, reconcile collected revenue with expected revenue, and identify discrepancies.

Objectives

• Calculate expected fares using route and passenger category.

• Sum expected and collected revenue.

• Report the difference and flag mismatched transactions.

Assumptions

• Each trip record contains route, passenger category, and collected fare.

• Base fares are Route A Rs. 20 and Route B Rs. 35.

• Students receive a 25% concession; senior passengers receive a 20% concession; other passengers pay full fare.

• Collected fares are supplied as sample transaction data.

Algorithm

1. Store route base fares and passenger records.

2. Select the base fare for each route.

3. Apply the category concession, if applicable.

4. Compare expected fare with collected fare for each passenger.

5. Accumulate expected and collected revenue and report the reconciliation difference.

Pseudocode

```
START
Set route fares and passenger transactions
FOR each passenger: select route base fare
Apply category concession
Compare expected fare with collected fare
Accumulate totals and report discrepancies
END
```

Python Program

```
base_fares = {"A": 20, "B": 35}
passengers = [
    ("A", "student", 15),
    ("B", "adult", 35),
    ("A", "senior", 16),
    ("B", "student", 26)
]
discounts = {"student": 0.25, "senior": 0.20, "adult": 0.0}
expected_total = 0
collected_total = 0

for route, category, collected in passengers:
    expected = base_fares[route] * (1 - discounts[category])
    expected_total += expected
    collected_total += collected
    status = "OK" if abs(expected - collected) < 0.01 else "MISMATCH"
    print(f"Route {route}, {category}: expected Rs.{expected:.2f}, "
          f"collected Rs.{collected:.2f} -> {status}")

print(f"Expected revenue: Rs.{expected_total:.2f}")
print(f"Collected revenue: Rs.{collected_total:.2f}")
print(f"Difference: Rs.{collected_total - expected_total:.2f}")
```

CASE STUDY 93 — OUTPUT AND VALIDATION

Sample Output

● Python Output — Sample Run

```
Route A, student: expected Rs.15.00, collected Rs.15.00 -> OK
Route B, adult: expected Rs.35.00, collected Rs.35.00 -> OK
Route A, senior: expected Rs.16.00, collected Rs.16.00 -> OK
Route B, student: expected Rs.26.25, collected Rs.26.00 -> MISMATCH
Expected revenue: Rs.92.25
Collected revenue: Rs.92.00
Difference: Rs.-0.25
```

Test Cases

Input / Condition	Expected Result
Collected fare equals expected fare	OK
Collected fare differs from expected fare	MISMATCH
Collected revenue is below expected revenue	Difference is negative

Flow Sequence

START → Input / initialize data → Process and validate → Apply decisions → Display results → END

Conclusion

The reconciliation compares calculated fares with collected amounts, totals the revenue, and flags individual discrepancies for review.